

Agentic Frameworks.

From single LLM calls to production-ready autonomous systems, built with LangChain, LangGraph & Langfuse.

TOPICS

Agents · Orchestration · Observability

STACK

Python · LangChain · LangGraph · Langfuse

MOTIVATION

Why do we need **agents**?

A single LLM call has a hard ceiling. Agents break through it.

Without agent

User: "Plan me a 5-day trip to Tokyo within \$2000."

LLM: "I'd suggest visiting Senso-ji temple, Shibuya... but I don't know *current* hotel prices, weather, or visa requirements. My knowledge has a cutoff."

→ Single-shot LLM can't access real-time data, call APIs, or correct itself.

With agent

Same query, but now with tools.

- calls `check_weather()`
- calls `search_hotels(budget=2000)`
- validates feasibility, loops if over-budget
- calls `build_itinerary()`
- returns grounded, day-by-day plan

→ **Agents reason** → **act** → **observe** → **adjust**.

DEFINITION

What is an **AI Agent**?

Autonomous software that perceives, reasons, and acts toward a goal.

*"AI agents are software systems that use AI to pursue goals and complete tasks on behalf of users. They show **reasoning, planning, and memory** and have a level of **autonomy** to make decisions, learn, and adapt."*

[↗ Google Cloud, What are AI agents?](#)

In modern systems, the **reasoning core is most often an LLM**. The concept itself predates LLMs and can also be powered by symbolic planners or other reasoners.

- **Perceive:** queries, environment state, tool outputs
- **Reason:** plan the next step
- **Act:** invoke tools, APIs, or other agents
- **Observe & iterate:** loop until the goal is met

TYPICAL COMPONENTS

Reasoner

Usually an LLM. The "brain"

Tools

APIs, functions, retrievers

Memory

Short & long-term state

Planner

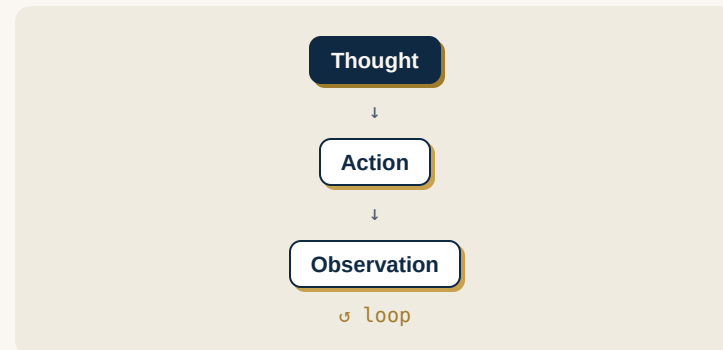
Routing & control flow

Common agent patterns.

Different "thinking rhythms" for different problems. ReAct is the default; others suit specific cases.

ReAct

REASON + ACT · [YAO ET AL. 2022](#)



Interleaves thought + action at every step. Best for **exploratory** tasks where next step depends on observations.

Plan-and-Execute

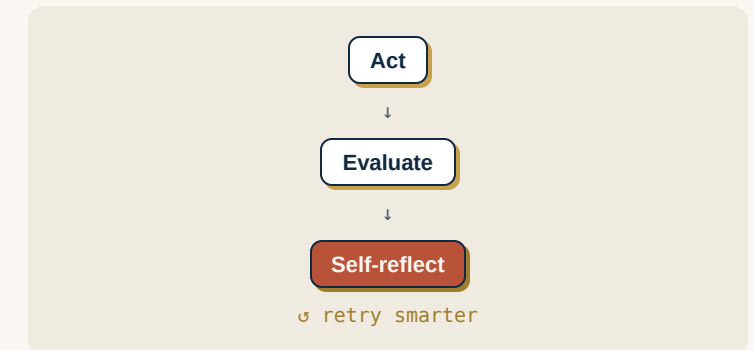
PLAN FIRST, THEN ACT · [WANG ET AL. 2023](#)



Decompose first, execute sequentially. Best for **well-defined multi-step** tasks. Fewer wasted LLM calls.

Reflexion

[SHINN ET AL. 2023](#)



Adds a self-critique loop. Best for **tasks with repeated failure modes** where the agent should learn from mistakes.

Also worth knowing: [Tree of Thoughts](#) (parallel reasoning paths with backtracking) for combinatorial problems, [ReWOO](#) (reasoning without observation, for cost-sensitive cases). All can be combined with multi-agent patterns.

Three tools, three layers.

Understanding how LangChain, LangGraph, and Langfuse fit together.

LangChain

THE BUILDING BLOCKS · [DOCS](#)

- Open-source framework for building LLM-powered applications (Python & JS/TS)
- Composable components: **chat models, prompts, tools, retrievers, memory, output parsers**
- Unified interface over hundreds of model providers and vector stores
- Released Oct 2022 by Harrison Chase; the foundation layer of the ecosystem

LangGraph

THE ORCHESTRATOR · [DOCS](#)

- Open-source framework built on LangChain for **stateful, multi-actor** applications
- Graph-based state machines: nodes + edges + shared state
- Supports **cycles, conditional branching, HITL, checkpointing**
- Production-grade in 2026; powers agents at Klarna, Uber, LinkedIn, Replit

Langfuse

THE ENGINEERING PLATFORM · [DOCS](#)

- Open-source **LLM engineering platform** (more than just observability)
- Covers: tracing, prompt management, evaluation, datasets, playground, metrics
- Framework-agnostic, self-hostable; OpenTelemetry-native
- YC W23; acquired by ClickHouse in 2026

Mental model: LangChain gives you the parts, LangGraph wires them into a working agent, Langfuse lets you observe and improve what's running.

PART ONE

LangChain.

The building blocks for composing LLM-powered applications from modular components.

A framework for LLM applications.

Compose modular components around any language model.

LangChain provides a **unified interface** over the messy ecosystem (model providers, vector stores, tool integrations, prompt templates) so you can swap any piece without rewriting the rest.

CORE COMPONENTS

ChatModel

PromptTemplate

Tool

Retriever

OutputParser

Memory

DocumentLoader

v1.0 heads-up: `AgentExecutor` (legacy) was moved to `langchain-classic` in [LangChain v1.0 \(Oct 22, 2025\)](#). Use `create_agent()` from `langchain.agents`, built on the LangGraph runtime.

```
from langchain.agents import create_agent

# v1.0 API: simpler, built on LangGraph runtime
agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool, query_tool],
    system_prompt="You are a helpful AI.",
)

response = agent.invoke({
    "messages": [{"role": "user",
                    "content": user_question}]
})
```

Provider-agnostic by design.

Swap models with one import. Test locally, deploy to production, keep the same code.

CHAT / COMPLETION MODELS

- **Cloud APIs:** OpenAI, Anthropic (Claude), Google (Vertex AI & GenAI), AWS Bedrock, Azure OpenAI
- **Local:** Ollama, vLLM, llama.cpp, LM Studio (load open-weight models from HuggingFace)

EMBEDDING MODELS

- **Cloud APIs:** OpenAI, Voyage, Cohere
- **Local:** Sentence-Transformers, HuggingFace embeddings (BGE, E5)

ONE-LINE SWITCHING · `init_chat_model()`

```
from langchain.chat_models import init_chat_model

llm = init_chat_model("anthropic:claude-sonnet-4-6")
# or init_chat_model("openai:gpt-5.4")
# or init_chat_model("ollama:llama3.1")
```

```
# Cloud chat model
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(
    model="claude-sonnet-4-6",
    temperature=0,
)

# Or local via Ollama
from langchain_ollama import ChatOllama
llm = ChatOllama(model="llama3.1", temperature=0)
```

```
# Local embeddings (free, runs on CPU/GPU)
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2",
    model_kwargs={"device": "cpu"},
)

# Or cloud embeddings
from langchain_voyageai import VoyageAIEmbeddings
embeddings = VoyageAIEmbeddings(model="voyage-3")
```


Retrieval, the backbone of RAG.

First-class integrations with every major vector store.

A **retriever** turns a query into a list of relevant documents. The building block for RAG pipelines and grounded agent responses.

VECTOR DATABASE SUPPORT

Pinecone

Qdrant

Milvus

Elasticsearch

MongoDB Atlas

Weaviate

pgvector

Chroma

Hybrid search (dense vector + BM25) often outperforms pure semantic search. Most stores support it natively via the `hybrid=True` flag.

```
retriever = ElasticsearchRetriever(
    es_url="http://localhost:9200",
    index_name="my-vector-index",
    embedding=embeddings,

    # Search params
    k=5,                    # Top K results
    fetch_k=20,            # Candidates before re-rank
    search_fields=["text", "title"],
    vector_query_field="text_vector",

    # Hybrid: dense + sparse
    hybrid=True,
    alpha=0.5,             # 0=BM25, 1=dense
)

docs = retriever.invoke("What is LangGraph?")
```

Chain → Graph.

Why linear pipelines aren't enough for autonomous agents.

	LANGCHAIN (LCEL)	LANGGRAPH
Topology	Linear, directed pipeline (input → A → B → output)	Graph with cycles , branches, conditionals
State handling	Implicit; values pass through pipe operators	Explicit shared state object, persisted via checkpoints
Retry / loop	Manual try/except, retry decorators	First-class; just route an edge back to a previous node
Human-in-the-loop	Not built in	Native <code>interrupt()</code> at any node
Best for	RAG, prompt chains, document Q&A, strict pipelines	Multi-step agents, retries, branching logic, multi-agent

Rule of thumb: if your agent needs to loop, retry, branch, or pause for human approval, reach for LangGraph. Otherwise LangChain alone is enough.

PART TWO

LangGraph.

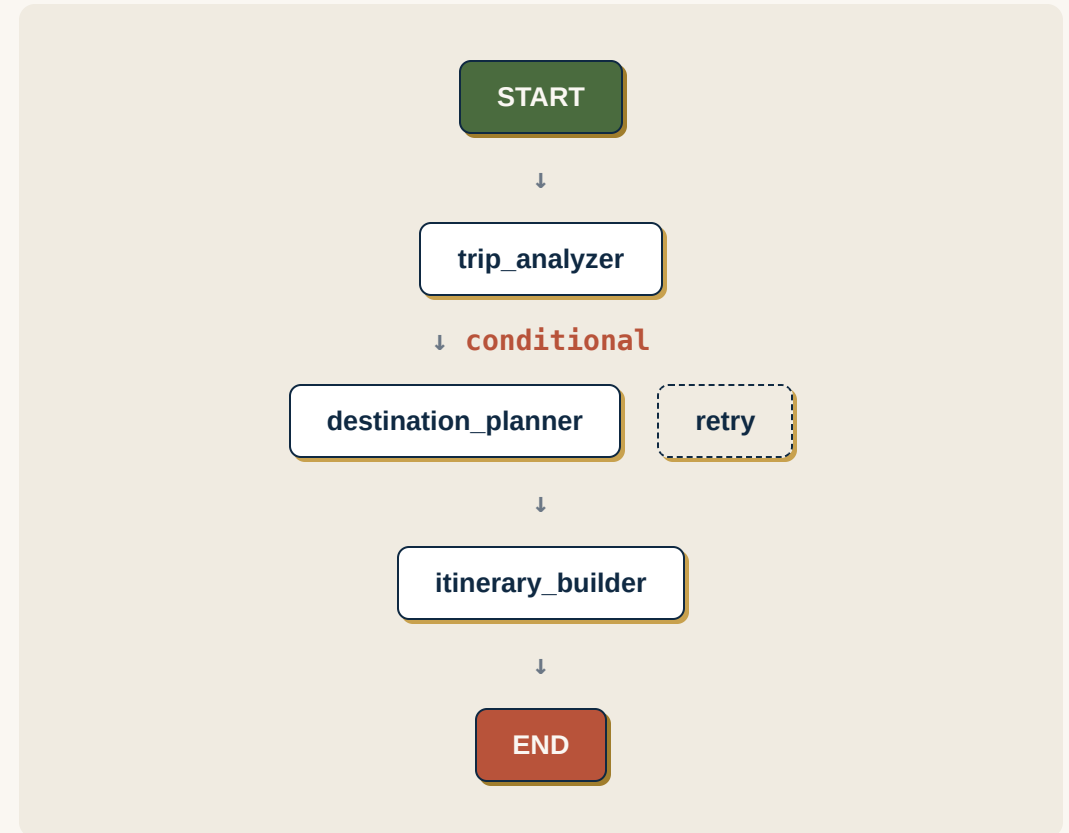
Stateful orchestration for multi-step, multi-agent workflows, modeled as a graph.

A graph-based **state machine**.

Code-first orchestration framework, with LangGraph Studio for visual debugging.

Define agent workflows as **nodes** (functions that process state), **edges** (transitions between nodes), and a **shared state schema** that flows through the graph.

- **Macro-orchestration:** high-level control over multi-agent flows
- Supports **branching, looping, multi-actor** systems naturally
- **Stateful:** persistence and checkpoints out of the box
- **Studio:** visual debugger; not a no-code builder



Why teams choose LangGraph.

From debugging to determinism: the production-grade benefits.

Explicit control flow

Nodes and edges make architecture instantly comprehensible. No more guessing what an agent will do next.

Stateful & long-running

Built-in checkpointers persist state to Postgres or Redis. Resume after crashes; time-travel debugging.

Deterministic execution

Same input + same state = same path. Critical for regression tests and reproducibility.

Faster root-cause analysis

When a tool call fails at step 7 of a 10-step agent, you can see *exactly* which node, with what input, in what state.

Modularity & reuse

Nodes are pure functions, easy to test in isolation, easy to compose into larger graphs.

Human-in-the-loop

Native `interrupt()`. Pause anywhere for human approval, then resume from exactly that point.

Demo: AI Travel Planner.

A three-node agent that builds a complete trip plan from a single user query.

What it does: analyzes user needs, validates feasibility, researches destinations, estimates costs, and produces a complete day-by-day itinerary with insider tips.

- **trip_analyzer:** understand user needs, validate trip feasibility (weather, visa, budget)
- **destination_planner:** research activities, hotels, restaurants for the chosen location
- **itinerary_builder:** organize everything into a day-by-day plan

SAMPLE INPUT

"Plan a 5-day cultural trip to Tokyo for 2 people in late March, budget \$3000."

TOOLS THE AGENT USES

check_weather

check_visa

estimate_budget

search_hotels

search_restaurants

validate_feasibility

Three primitives. That's all.

Everything in LangGraph reduces to State, Nodes, and Edges.

State

THE DATA

A typed schema (TypedDict, Pydantic, or dataclass) shared between nodes. Holds messages, tool outputs, intermediate results.

Node

THE WORK

A function that reads state, does work (calls an LLM, invokes tools, transforms data), and returns updates.

Edge

THE CONTROL

Defines what node runs next: fixed transitions, or conditional routing based on the current state.

Coming up: we'll build each primitive step-by-step using the Travel Planner example, then wire them together into a runnable graph.

Nodes: where work happens.

Sync or async functions that read state and return updates.

A node is just a Python function. It receives the current graph state (optionally `config` and `runtime`), does its work, and returns a dict of state updates.

- Handles **one discrete task**: LLM call, tool execution, data transform
- Can be **sync or async**
- Returns **partial state updates**, not the full state
- Registered via `builder.add_node(name, fn)`

```
builder = StateGraph(State)

def plain_node(state: State):
    return state

def node_with_runtime(state: State, runtime: Runtime[Context]):
    print("user_id:", runtime.context.user_id)
    return {"results": f"Hello, {state['input']}!"}

def node_with_config(state: State, config: RunnableConfig):
    thread_id = config["configurable"]["thread_id"]
    return {"results": f"Hello, {state['input']}!"}

builder.add_node("plain_node", plain_node)
builder.add_node("with_runtime", node_with_runtime)
builder.add_node("with_config", node_with_config)
```


Defining the agent.

Tools + system prompt + create_agent → an LLM that can call functions.

The **trip_analyzer** node wraps a ReAct-style agent. Inside the node, we:

- Define the **tools** this agent can call
- Write a focused **system prompt** for its job
- Use `create_agent()`, the modern replacement for `AgentExecutor`

Pattern: each node owns its agent. Tools are scoped to that node's job, keeping prompts focused and reducing wrong tool selection.

```
def trip_analyzer_node(state: TravelState) -> dict:
    """Analyze user query and validate feasibility."""
    user_query = state["user_query"]

    # Tools available to this agent
    tools = [
        parse_travel_details,
        check_weather,
        check_travel_requirements,
        estimate_budget,
        validate_trip_feasibility,
    ]

    # Focused system prompt
    system_prompt = """You are a Trip Analyzer agent.
    Parse the user's request, check feasibility,
    and produce a structured trip_analysis."""

    # Modern API, replaces deprecated AgentExecutor
    from langchain.agents import create_agent
    agent = create_agent(model=llm, tools=tools)
```

Invoking the agent, with tracing.

One callback, full observability. Every LLM call, every tool call, captured.

The Langfuse `CallbackHandler` is the single integration point. Pass it once in `config`, and the entire execution tree (LLM calls, tool invocations, intermediate thoughts) is automatically traced.

- No code changes inside tools or prompts
- Nested calls auto-linked into a hierarchy
- Latency, token usage, errors, all captured

This is the magic: instrument once at the agent level, get full visibility everywhere underneath.

```
from langfuse.langchain import CallbackHandler
from langchain_core.messages import SystemMessage, HumanMessage

# Single integration point, traces everything below
langfuse_handler = CallbackHandler()

result = agent.invoke(
    {
        "messages": [
            SystemMessage(content=system_prompt),
            HumanMessage(
                content=f"Analyze this travel query: {user_query}"
            ),
        ]
    },
    config={"callbacks": [langfuse_handler]},
)

# `result` now contains the agent's reasoning + tool outputs
trip_analysis = result["messages"][-1].content
```

Returning state updates.

Nodes return partial state. LangGraph merges it into the shared object.

A node never mutates state directly. It returns a dict, and LangGraph applies it to the shared state using **reducers** (e.g., append for lists, replace for scalars).

- **Scalar fields** (str, dict) → replaced on update
- **List fields** (with `Annotated[list, add]`) → appended
- The next node sees the merged state automatically

Best practice: always log **what** the node did into `thoughts` and `actions`.
Invaluable for debugging.

```
return {
    "trip_analysis": trip_analysis,
    "current_node": "destination_planner",
    "thoughts": state["thoughts"] + [
        "Trip analysis completed"
    ],
    "actions": state["actions"] + [
        {
            "node": "trip_analyzer",
            "action": "analyze_trip",
            "timestamp": datetime.now().isoformat(),
        }
    ],
}
```

Edges: how the graph routes.

Four types. Together they cover every control-flow pattern you need.

Normal Edge

Always go from node A → node B. Fixed, unconditional. The simplest case.

Entry Point

The first node to run when the graph receives input. Special edge from `START`.

Conditional Edge

Call a routing function to decide which node(s) come next. Can also terminate the graph.

Conditional Entry Point

Start at different nodes based on input (e.g., route by query type or user tier).

Composition rule: a node can have multiple outgoing edges of different types. The graph engine evaluates them and routes accordingly.

Normal edge.

Unconditional A → B transitions.

After `node_a` finishes, always run `node_b`. No conditions, no routing logic.

Use this for **guaranteed sequential steps** (e.g., always log results after the LLM call, always validate after parsing).

Tip: if a node should always be followed by the same next node, prefer a normal edge. Conditional edges add complexity you don't need here.

```
graph.add_edge("node_a", "node_b")
```

node_a

↓ always

node_b

Conditional edge.

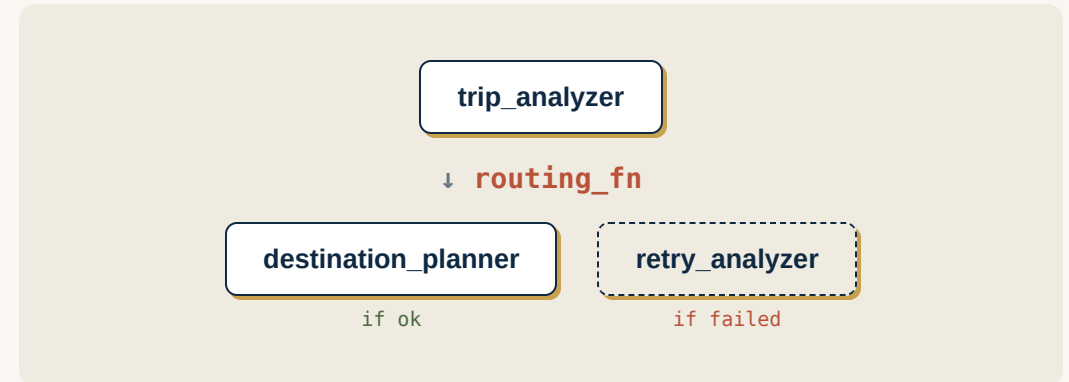
Route to one of many nodes (or terminate) based on state.

Call a **routing function** after the source node finishes. The function reads state and returns the next node's name (or `END`).

```
def routing_function(state):  
    if state["analysis_ok"]:  
        return "destination_planner"  
    return "retry_analyzer"
```

```
graph.add_conditional_edges(  
    "trip_analyzer",  
    routing_function,  
)
```

```
# Or with explicit mapping  
graph.add_conditional_edges(  
    "node_a",  
    routing_function,  
    {True: "node_b", False: "node_c"},  
)
```



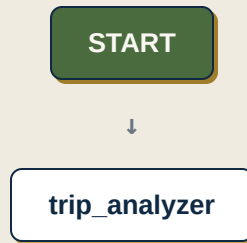
Entry points.

Where the graph begins. Fixed, or chosen dynamically.

FIXED ENTRY POINT

Always start at the same node when the graph receives input.

```
from langgraph.graph import START
graph.add_edge(START, "trip_analyzer")
```

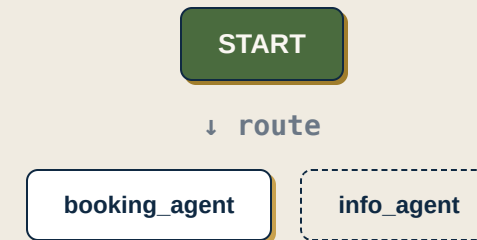


CONDITIONAL ENTRY POINT

Choose a starting node based on the incoming input (e.g., query type, user role).

```
def route_by_intent(state):
    if state["intent"] == "book":
        return "booking_agent"
    return "info_agent"

graph.add_conditional_edges(
    START, route_by_intent,
)
```



State: the shared memory.

A typed schema that flows through every node, and persists across executions.

State is the **shared data structure** passed between nodes. It defines the graph's input/output schema and, via checkpoints, survives across runs.

- Defined as `TypedDict`, `Pydantic`, or `dataclass`
- Stores messages, intermediate results, metadata, tool outputs
- Nodes read from it and return updates to it
- Persisted via **checkpointers** (Postgres/Redis/in-memory)

Use `Annotated` with a reducer for fields that should accumulate (e.g., `messages: Annotated[list, add_messages]`).

```
from typing import Optional
from typing_extensions import TypedDict
from langchain_core.messages import AnyMessage
```

```
class State(TypedDict):
    messages: list[AnyMessage]
    extra_field: int
```

```
# Or with Pydantic for runtime validation
from pydantic import BaseModel
```

```
class StateModel(BaseModel):
    messages: list[AnyMessage] = []
    extra_field: int = 0
```


Designing a real state schema.

Group fields by node responsibility. Track everything you'll want to debug later.

```
class TravelState(TypedDict):
    """Shared state for the AI travel planner graph."""

    # ===== USER INPUT =====
    user_query: str          # Original travel request from user

    # ===== TRIP ANALYSIS (Node 1: Trip Analyzer ReAct Agent) =====
    trip_analysis: Optional[dict]  # Output from Trip Analyzer

    # ===== DESTINATION PLANNING (Node 2: Destination Planner ReAct Agent) =====
    destination_plan: Optional[dict]  # Output from Destination Planner

    # ===== ITINERARY (Node 3: Itinerary Builder Standard Node) =====
    itinerary: Optional[dict]  # Final day-by-day itinerary

    # ===== ReAct TRACKING (for ReAct agents in nodes 1 & 2) =====
    thoughts: list[str]        # All thoughts from ReAct reasoning
    actions: list[dict]         # [{node, tool, params, timestamp}]
    observations: list[dict]    # [{action_id, result}]

    # ===== ROUTING =====
    current_node: str          # Which node is currently active
```

Compiling & running the graph.

From State + Nodes + Edges → a callable agent.

Once the three primitives are defined, **compile** the graph into a runnable, then `invoke` (sync) or `astream` (streaming).

- `.compile()` validates & produces an executor
- `.invoke()` runs to completion, returns final state
- `.stream()` yields intermediate state updates
- `.get_graph().draw_mermaid()` renders the diagram

Same pattern, every graph. Build → Compile → Invoke. Once you know this, you can read any LangGraph codebase.

```
from langgraph.graph import StateGraph, START, END

# 1. Build
builder = StateGraph(TravelState)
builder.add_node("trip_analyzer", trip_analyzer_node)
builder.add_node("destination_planner", destination_planner_node)
builder.add_node("itinerary_builder", itinerary_builder_node)

builder.add_edge(START, "trip_analyzer")
builder.add_edge("trip_analyzer", "destination_planner")
builder.add_edge("destination_planner", "itinerary_builder")
builder.add_edge("itinerary_builder", END)

# 2. Compile
graph = builder.compile()

# 3. Invoke
result = graph.invoke({
    "user_query": "5-day cultural trip to Tokyo, $3000 budget",
    "thoughts": [], "actions": [], "observations": [],
})

print(result["itinerary"])
```

Production-grade features.

Three capabilities that separate LangGraph from a simple agent loop.

Checkpointers

PERSISTENCE

Save state to Postgres, Redis, or SQLite after every node.
Resume after crashes; **time-travel** to replay from any step.

```
from langgraph.checkpoint.postgres \
    import PostgresSaver

checkpointer = PostgresSaver(...)
graph = builder.compile(
    checkpointer=checkpointer
)
```

Human-in-the-Loop

APPROVAL WORKFLOWS

`interrupt()` pauses execution mid-graph, waits for human approval, then resumes from exactly that point.

```
from langgraph.types import interrupt

def review_node(state):
    decision = interrupt({
        "draft": state["itinerary"]
    })
    return {"approved": decision}
```

Streaming

REAL-TIME UI

Stream LLM tokens, intermediate state, or node-level events for responsive frontends.

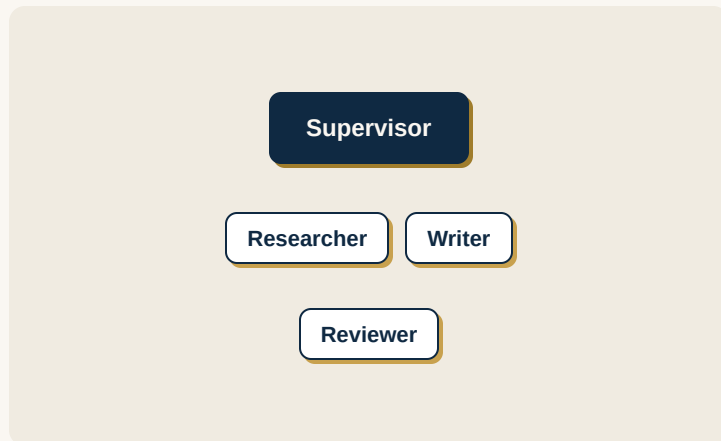
```
async for chunk in graph.astream(
    {"user_query": "..."},
    stream_mode="updates",
):
    print(chunk)
    # → live UI updates
```

When one agent isn't enough.

Three common patterns for coordinating multiple agents.

Supervisor

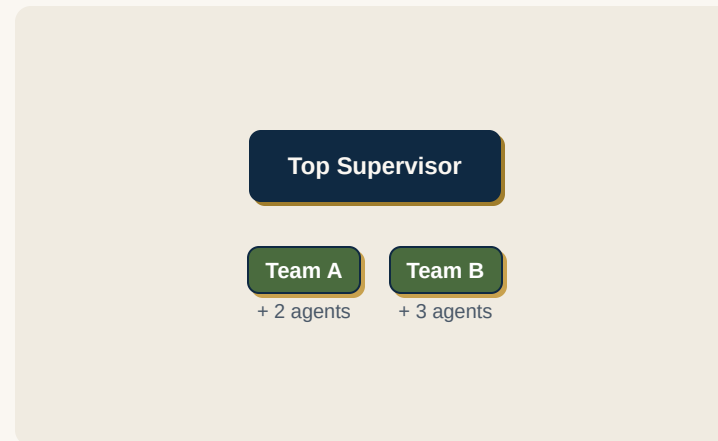
HUB & SPOKE



One LLM picks which worker runs next, like a project manager.

Hierarchical

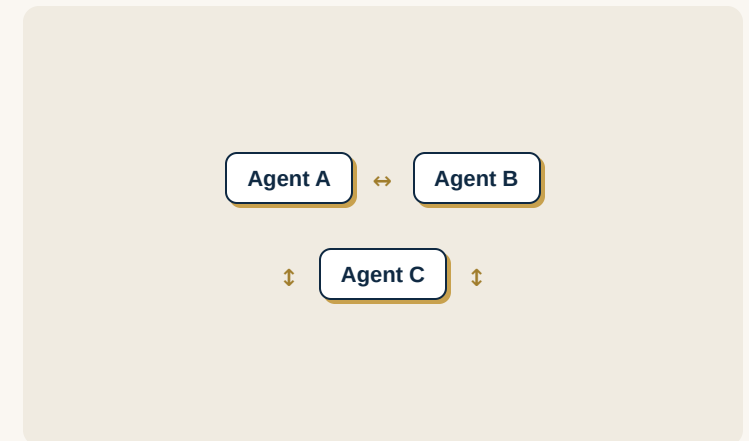
NESTED TEAMS



Supervisors all the way down. Scales to large org-chart agents.

Swarm

PEER HAND-OFF



Agents hand off to each other directly. No central coordinator.

PART THREE

Langfuse.

Observability for non-deterministic systems. Trace, evaluate, and improve your agents in production.

Why agents need **observability**.

Traditional logging breaks down on non-deterministic, multi-step systems.

*When an agent fails on step 7 of a 10-step run, "**print debugging**" doesn't scale. You need to see prompts, responses, tool calls, and timing across the entire execution tree.*

Langfuse is an **open-source LLM engineering platform**. It's model and framework agnostic, self-hostable, with first-class integrations for LangChain and LangGraph.

- Traces capture **every** LLM call, tool call, and retrieval
- Sessions and users group related traces
- Built-in evaluation, prompt versioning, datasets

WHAT GETS CAPTURED

Prompts sent & received

Tool calls & outputs

Token usage per model

Latency at each step

Errors & retries

Cost per request

Self-hostable: prompts and responses stay on your infrastructure (important for regulated data).

End-to-end tracing.

One callback, full visibility across every LLM call, tool, and node.

- **Hierarchical traces:** nested calls automatically linked into a tree
- **Captures everything:** prompts, responses, tool I/O, errors, latency, tokens
- **Integrates seamlessly** with LangChain and LangGraph callbacks
- **OpenTelemetry-compatible:** works with existing monitoring stacks

Production tip: add `user_id` and `session_id` to every trace. Lets you debug exactly what one user experienced.

TRACE HIERARCHY

▼ Agent workflow (2.19s)

▼ trip_analyzer (1.04s · 70 tokens)

- LLM: gpt-4o-mini
- tool: check_weather
- tool: estimate_budget

▼ destination_planner (1.15s · 114 tokens)

- LLM: gpt-4o-mini
- tool: search_hotels

▼ itinerary_builder (0.20s)

Six capabilities, one platform.

Beyond tracing: the full lifecycle of LLM application development.

Observability

Trace every request end-to-end: prompts, responses, tool calls, errors. The foundation.

Metrics

Latency, token usage, cost. Aggregated by model, user, route, environment.

Prompt Management

Version prompts centrally. Link versions to traces. Deploy without redeploying code.

Playground

Compare models & prompts side-by-side. Iterate on prompts using real production data.

Evaluation

LLM-as-a-judge, programmatic evals, dataset experiments. Catch quality drift before users do.

Annotations

Human review queues. Score outputs, flag issues, build labeled datasets for fine-tuning.

From `@observe()` to dashboards.

Drop-in tracing for any function, then aggregate into actionable dashboards.

```
from langfuse import observe
from langfuse.openai import openai

# Drop-in wrapper, adds OpenTelemetry
# tracing to OpenAI. Many integrations
# available (LangChain, Anthropic, etc.)

@observe() # decorate any function;
          # nested calls auto-linked
def handle_request(text: str) -> str:
    res = openai.chat.completions.create(
        model="gpt-5",
        messages=[
            {"role": "system",
             "content": "Summarize in one sentence."},
            {"role": "user", "content": text},
        ],
    )
    return res.choices[0].message.content
```

WHAT YOU SEE IN THE UI

🕒 **Traces by time:** volume trends

💰 **Cost by model:** spend breakdown

👤 **User consumption:** top users, top costs

🎯 **P95 latency:** slow tails by route

⚠️ **Error rates:** failures & retries

Custom metrics: the Langfuse API supports SQL-like aggregations (e.g.,
`SUM(totalCost) GROUP BY userId` for the last 30 days).

Prompts as **versioned artifacts**.

Iterate without redeploying. Compare models without writing code.

Prompt Management

Store prompts in Langfuse. Fetch the latest `production` version at runtime. No code change to update a prompt.

```
from langfuse import get_client

langfuse = get_client()

# Fetches latest 'production' version
# Cached client-side, revalidates async
prompt = langfuse.get_prompt(
    "support-reply", type="chat"
)

compiled = prompt.compile(
    tone="friendly", input=user_input
)

res = openai.chat.completions.create(
    model="gpt-5",
    messages=compiled,
)
```

Playground

A side-by-side UI for comparing prompts and models. Load any trace into the playground to iterate on it.

openai: gpt-4.1
"You are a helpful assistant."
→ Asks 3 clarifying questions before planning

openai: gpt-5
"You are a travel planner..."
→ Produces a 5-day itinerary immediately

Compare outputs · A/B test prompts · Save winners as versions

Catch quality drift before users do.

Automated evals + human review = a feedback loop you can actually trust.

Evaluation

Run datasets through your pipeline, score outputs programmatically or with LLM-as-a-judge, and track scores over time.

```
dataset = langfuse.get_dataset(
    "capital_cities"
)

def task(*, item, **_):
    return run_my_agent(item["input"])

def accuracy_eval(*, output,
                  expected_output, **_):
    ok = expected_output.lower() \
        in output.lower()
    return Evaluation(
        name="accuracy",
        value=1.0 if ok else 0.0,
    )

result = langfuse.run_experiment(
    name="agent-v2",
    data=dataset.items,
    task=task,
    evaluators=[accuracy_eval],
)
```

Annotations

Human review queues for traces flagged by users, evals, or sampling. Build labeled datasets to drive improvements.

MAIN ANNOTATION QUEUE

Trace #b13e4216 · User u-HLd...

core-issue

resolved

accuracy

true

answer_correct

false

tonality_correct

true

→ Failed examples become next iteration's eval set.

This isn't a **toy** framework.

Production deployments running at scale, today.

*"Average resolution time fell from **11 minutes to 2 minutes**. The system now manages **2.5 million conversations**, delivering output comparable to **700 full-time employees**."*

Klarna · customer support, deployed on LangGraph · [↗ LangChain case study](#)

80%

Reduction in resolution time. Klarna customer support after migrating to LangGraph agents

2×

Response accuracy improvement. AppFolio Realm-X copilot, LangChain → LangGraph migration

2%

False positive rate. Rexera (real-estate QC), down from 8% with CrewAI (and 35% with raw single-prompt LLMs)

Sources: [↗ LangChain Klarna case study](#) · [↗ Klarna press release](#) · [↗ AppFolio case study](#) · [↗ Rexera case study](#)

When to use what?

A practical guide for picking the right tool for your problem.

IF YOUR PROBLEM IS...	REACH FOR	WHY
Simple RAG pipeline	LangChain (LCEL) alone	Linear flow, no branching. LCEL is faster to write & easier to debug.
Single-tool ReAct agent	<code>create_agent()</code> from LangChain	Modern replacement for AgentExecutor. Enough for 1–3 tools, linear execution.
Multi-step agent with loops, retries, or branching	LangGraph	Cyclic graphs handle this natively. Linear chains can't.
Need human approval mid-execution	LangGraph with <code>interrupt()</code>	HITL is a first-class citizen. Pause, get approval, resume from exact state.
Multi-agent system (research, coding, etc.)	LangGraph (Supervisor pattern)	Explicit control flow keeps it debuggable. CrewAI is faster to start but less control.
Anything in production	+ Langfuse	You can't fix what you can't see. Mandatory for non-toy systems.

WRAP-UP

Next *steps.*

Explore AI agents through real projects, and bring observability from day one.

HANDS - ON

- Build the travel planner end-to-end
- Add a retry loop with conditional edge
- Wire in Langfuse, inspect the trace

EXTEND

- Add Postgres checkpointer
- Insert a human approval step
- Try a Supervisor multi-agent

REFERENCE

- python.langchain.com
- langchain-ai.github.io/langgraph
- langfuse.com/docs

"You can't fix what you can't see."

· END ·